# Learning to Fuse: A Data-Driven Cost Model for TorchInductor

Alan Luo
Sameer Narendran
alanluo3@illinois.edu
sameern3@illinois.edu

## Abstract

Operator fusion is a critical optimization in modern deep learning compilers, enabling reduced memory traffic and kernel launch overhead on GPUs. In TorchInductor, fusion decisions are currently governed by a hand-crafted heuristic scoring function that encodes static, hardware-agnostic rules. While effective for common model architectures and hardware configurations, this approach struggles to generalize across diverse neural network topologies and evolving GPU architectures, and it fails to account for global graph-level trade-offs. In this work, we propose a learned fusion decision engine that replaces heuristic scoring with a data-driven cost model. Our approach enables hardware-aware, globally informed fusion decisions without increasing compilation overhead. Experimental evaluation across a wide range of Transformer-based models demonstrates that the learned scheduler achieves performance parity with TorchInductor's existing heuristics, while incurring only negligible additional compilation time. These results validate learned fusion as a viable and extensible alternative to heuristic-based compiler optimization.

## 1 Introduction

The efficiency of modern deep learning compilers, such as TorchInductor, hinges on their ability to minimize the overhead associated with memory bandwidth and kernel launches. This is primarily achieved through operator fusion—the process of combining multiple logical operations into a single GPU kernel. Within the TorchInductor architecture, the decision of whether to fuse two nodes is dictated by a scoring mechanism in the `score_fusion` function. Currently, this mechanism relies on a set of hand-written heuristics that assign a priority score to fusion candidates based on static parameters, such as the number of memory-bound operations, node proximity, and binary flags like `should_prefer_unfused_mm`.

While these heuristics have been finely tuned for standard architectures like ResNet or Transformers on specific hardware, they suffer from three critical flaws that motivate the need for a learned approach:

1. **Hardware-Agnostic Static Logic:** Human-written heuristics are inherently limited by the developer's ability to model the hardware. A heuristic might favor fusion to reduce memory traffic, but it cannot accurately predict if the resulting kernel will suffer from high register pressure or occupancy issues on a specific NVIDIA architecture (e.g., Ampere vs. Blackwell).
2. **Heuristic Rigidity:** The existing `if-else` logic in `score_fusion` cannot scale with the non-linear data flows of emerging architectures. This hard-coded approach creates a performance ceiling, as it is unable to recognize or exploit novel fusion patterns that fall outside its predefined rule set.
3. **Greedy Optimization Limits:** Current scoring evaluates fusion candidates in isolation, ignoring how local decisions constrain global graph efficiency. A data-driven model utilizes graph-level features to identify long-range dependencies, allowing the compiler to bypass sub-optimal local merges in favor of higher-gain global execution plans.

Therefore, the objective of this project is to replace the current heuristic-based scoring mechanism with a data-driven cost model. By training an embedding model on ground-truth execution latencies from a large-scale dataset of intermediate graph representations, we transform fusion selection into an objective optimization problem. This approach enables the compiler to adapt to specific GPU hardware characteristics and varying neural network topologies, maximizing execution efficiency relative to theoretical hardware limits.

## 2 Background

Research in machine learning–based compilation has evolved from heuristic-driven autotuning toward learned representations that directly model program behavior and hardware interaction. Early work framed compiler optimization as a black-box search problem, while more recent approaches view programs and compiler traces as structured data amenable to representation learning. Our work extends the learning-based compiler optimization paradigm introduced in [4] by treating compiler execution traces as a structured language, allowing models to learn compositional patterns that directly impact hardware utilization and execution efficiency.

**Autotuning and Cost Modeling:** Early frameworks like TVM [1] and Halide utilized search-based autotuning and simple regressors to explore schedule spaces. However, these

analytical models often fail to capture stochastic GPU hardware behaviors, such as cache contention and register pressure. Substituting these fixed heuristics with learned cost models has been shown to significantly reduce the performance gap between automated and manual kernel engineering [4]. Ithemal [5] demonstrated that neural networks could accurately predict throughput for basic blocks of x86 assembly, outperforming heuristic models.

**Graph Representations:** While models like CodeBERT [2] capture local semantics, compiler optimization problems are fundamentally graph-structured. Control-flow graphs, data-flow graphs, and dependency graphs encode relationships that are not well-represented by linear token sequences. Graph Neural Networks (GNNs), and in particular Graph Attention Networks (GATs) [7], have demonstrated strong performance in modeling relational inductive biases by selectively aggregating information from neighboring nodes. In the compiler domain, graph-based representations enable models to reason about operator dependencies, memory access patterns, and scheduling constraints. By utilizing GNNs, we can compute embeddings that aggregate information from neighboring operators, providing the structural context necessary to predict the performance impact of a fusion cluster [6].

**Representation Learning for IR:** To move beyond manual feature engineering, we treat compiler Intermediate Representations (IR) as a structured language. Pre-trained encoders like CodeBERT [2] capture global semantics from `torch.fx` graph traces, mapping them to high-dimensional latent spaces. To refine these for performance prediction, we utilize the SimCSE contrastive learning framework [3]. This allows the model to identify structural invariants that dictate execution costs, leveraging latent space distance to represent the performance delta between fusion strategies [6].

## 3 Design & System Overview

The system architecture transitions the operator fusion decision process from a static, rule-based heuristic into a dynamic, graph-aware neural evaluation engine. By combining a Transformer-based semantic encoder with a Graph Neural Network (GNN) structural contextualizer, the system internalizes the complex, non-linear relationship between computation graph topology, operator dependencies, and hardware execution efficiency. This design enables the compiler to make fine-grained, hardware-aware fusion decisions, effectively bridging the gap between static heuristics and performance-informed optimization.

### 3.1 Architectural Components

The design consists of three primary stages:

1. **Semantic Encoding:** The first stage leverages a pre-trained Transformer encoder, CodeBERT, to ingest serialized `torch.fx` IR representations. This model was then fine-tuned on our IR representations. Each operator in the graph is linearized and tokenized, mapping raw computation traces into a dense, high-dimensional latent space ($\mathbb{R}^{768}$).
   This embedding captures operator semantics, operand types, and high-level functional behavior. To ensure robustness to variable naming and literal values, a normalization pass standardizes operator identifiers and maps numerical constants to a `<NUM>` token. This step reduces the vocabulary size, prevents overfitting on irrelevant syntactic details, and ensures that semantically equivalent IR subgraphs map to similar latent representations.
   This stage also supports modular extensibility: new operator types can be incorporated without retraining the entire model, as long as their tokenization follows the same normalization schema.
2. **Structural Contextualization:** The second stage employs a Graph Attention Network (GATv2) to contextualize the embeddings across the computation graph. Unlike static aggregators, GATv2 introduces dynamic attention weights for each edge, allowing the model to learn which dependencies are most critical for predicting performance. For instance, the model can assign higher weight to edges where register pressure or memory access patterns significantly influence latency.
   During message passing, each node's embedding is recursively updated by aggregating information from its neighbors, capturing both local and global structural context. By explicitly encoding topological and dependency information, this stage allows the system to reason about interactions that sequence-based models cannot capture, such as parallelism potential and memory coalescing in fused subgraphs.

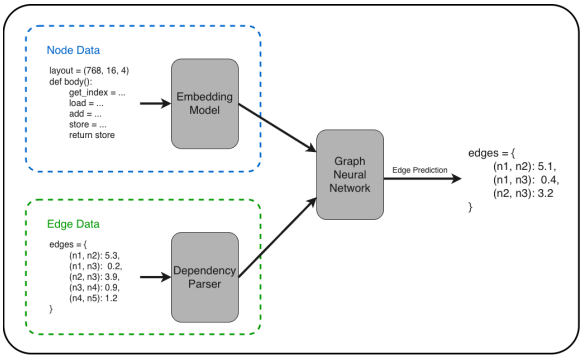


**Figure 1.** Our architecture combines a fine-tuned embedding model (based on CodeBERT) and a Graph Neural Network to predict edge weights. Higher weights indicates higher fusion priority.

3. **Neural Scoring Engine:** The final fusion decision is generated by an Edge MLP, which evaluates candidate fusion pairs. For each edge connecting a source node $h_{src}$ and a destination node $h_{dst}$, the MLP receives a concatenated vector of their embeddings along with a small edge attribute vector encoding metadata such as tensor shape, broadcast patterns, or memory layout differences.
   This module regresses a hardware-aware utility score, effectively predicting the relative performance benefit of applying a fusion transformation. Unlike heuristic approaches that rely on fixed rules, this scoring engine adapts to different hardware targets, kernel shapes, and IR configurations. The output scores are used to prioritize fusion actions during compilation, enabling the compiler to select transformations that maximize throughput and resource utilization.

### 3.2 Contrastive Embedding Learning

To train the model to produce meaningful embeddings of the IR representations, we employ a contrastive learning approach using the SimCSE framework[3]. Negative pairs are formed by sampling unrelated IR graphs from the training corpus to encourage the model to distinguish between semantically different structures. The model is trained to minimize the distance between embeddings of similar graphs while maximizing the distance from dissimilar ones. This process enhances the model's ability to capture nuanced semantic relationships in the IR space.

### 3.3 Ranked Pair Performance Alignment

The fusion scoring engine is trained using a ranked pair loss function. For each training instance, we generate pairs of candidate fusion transformations along with their performance metrics (e.g., execution time, memory consumption, operation type). The model is trained to predict higher scores for transformations that yield better performance outcomes. This ranking-based approach allows the model to learn relative performance differences, which is more robust than absolute regression in the presence of noise and variability in hardware measurements and model architectures.

### 3.4 Data Acquisition Strategies

To overcome data scarcity, we utilize a dual-stream strategy:

1. **Intermediate Representation Extraction:** We instrumented TorchInductor with flags such as `TORCH_COMPILE_DEBUG` and `TRITON_SAVE_TTIR` to export IR traces and fusion scores calculated on model compilation. This data was used for the initial training of our Graph Neural Network.
2. **Benchmarking:** We used TorchInductor's kernel generation and benchmarking functions such as `benchmark_combo_kernel` and `speedup_by_fusion` to time speedups of node fusions. This data was used for fine-tuning our Graph Neural Network.

### 3.5 Inference and Integration

The system is deployed as an inference service within the compiler toolchain. A Flask-based GNN server is responsible for embedding and scoring candidate subgraphs. During compilation, TorchInductor serializes each candidate subgraph into the normalized IR format and transmits it to the server.

Upon receiving the request, the server performs semantic encoding via CodeBERT, structural contextualization with GATv2, and edge scoring through the MLP. The returned performance scores guide the compiler's fusion decisions, replacing the legacy greedy heuristic with an objective, hardware-aware prediction.

This design enables online inference at compilation time without requiring kernel recompilation, and supports integration into distributed or cloud-based compilation pipelines. By decoupling model inference from the compiler process, the architecture maintains flexibility, scalability, and compatibility with multiple backends or hardware targets.

## 4 Implementation

We implemented our architecture using PyTorch and PyTorch Geometric. Training was performed on an Amazon EC2 g5.2xlarge instance with a single NVIDIA A10G GPU. The CodeBERT encoder was fine-tuned from the pre-trained `microsoft/codebert-base` checkpoint, while the GATv2 layers and Edge MLP were trained from scratch. Our training data was gathered from 156 different models from a wide range of text model families including: BERT, GPT, T5, BART, XLNet, XLM, Phi, RWKV, Bloom, and LLaMA. Our embedding model was tuned on 18k node serializations. Our GNN was trained on 299 graphs, made up of 18k node representations and 30k valid fusions. The trained models and training parameters used can be found in Appendix A.

Modifications to TorchInductor's scheduler were made to generate training data so that every fusion pass performed by the scheduler was captured and exported by editing the `get_possible_fusions`, and `benchmark_kernel_get_times` functions. The `score_fusion` function was replaced to query the GNN inference server for real-time fusion decisions during compilation.

Our full code can be found at github.com/sameer-n012/pytorch-learned-fusion.

## 5 Evaluation

### 5.1 Evaluation Setup

To evaluate the effectiveness and correctness of our GNN-based fusion decision engine, we conducted experiments on the same Amazon EC2 g5.2xlarge instance used for training. We selected a diverse set of benchmark models from

the Hugging Face model hub including BERT, GPT, XLNet, MiniLM, and others, ensuring coverage across various architectures and sizes. Our GNN inference server was run on the evaluation machine.

Each model was compiled using TorchInductor with both the legacy heuristic fusion scheduler and our GNN-based fusion scheduler. In addition, we implemented a baseline that uses random fusion rankings to provide a lower bound on performance. Each model was compiled with a warm-up phrase to measure the initial compilation time, followed by 10 iterations of inference to capture steady-state performance metrics. This was repeated 10 times and averaged. We recorded the average compilation time and average inference latency for each configuration.

### 5.2 Evaluation Results

Our evaluation demonstrated that our GNN-based fusion decision engine performed comparably to the heuristic-based function, achieving similar inference latencies across the benchmark models. Some models exhibited marginal improvements in latency, while others showed slight regressions.

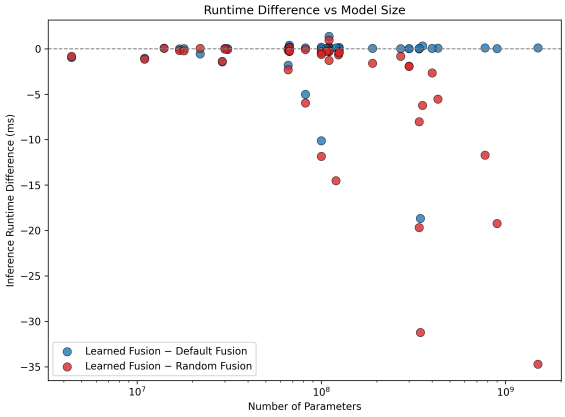


**Figure 2.** The inference latency difference between our learned fusion method and the default TorchInductor method (blue) and the random fusion baseline (red) across 48 different models.

In particular, Figure 2 shows 3 models with significant improvement in runtime latency compared to the default fusion strategy: DeBERTa Large (18ms), DeBERTa Base (10ms), and DistilGPT2 (5ms). The DeBERTa variants in particular had many fusion optimization passes compared to other model families, with the base model having 8 fusion passes and the large model having 9 fusion passes. This indicates that the GNN-based approach may be particularly effective in scenarios where multiple fusion opportunities exist, allowing it to make more nuanced decisions that lead to better overall performance. Overall, the GNN-based approach maintained

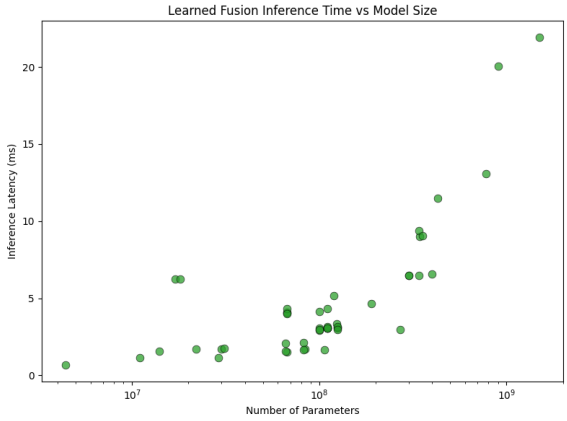


**Figure 3.** The inference latency of our learned fusion method across 48 different models.

performance parity with the heuristic method, validating its effectiveness in making fusion decisions. Similar to the heuristic-based approach, the runtime latency of the models produced by our learned fusion method grows linearly with respect to the number of parameters in the model, as shown in Figure 3.

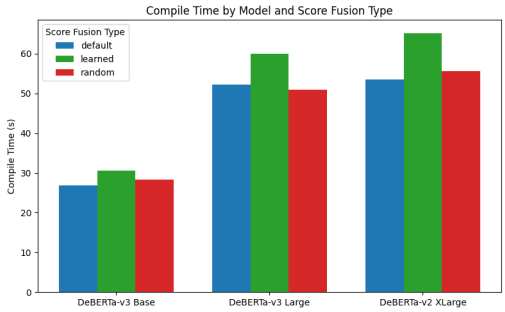


**Figure 4.** The compile time for 3 models from the DeBERTa family. Each model has 190M, 430M, and 900M parameters respectively.

In addition, we observed that there was only a very slight increase in compilation time when using the GNN-based scheduler compared to the heuristic approach, despite the additional overhead of querying the inference server, as shown in Figure 4. This indicates that the majority of the compilation time is dominated by other factors, such as kernel generation and optimization, rather than the fusion decision process itself. It also suggests that the GNN-based approach is efficient enough to be integrated into the compilation pipeline without introducing significant latency.

Our comparison against random fusion benchmarks further highlights the effectiveness of both the heuristic and GNN-based approaches. The random fusion strategy resulted

in a substantial increase in inference latency for large models, as seen in Figure 2, with increases of up to 35 ms for models like GPT2 XL, GPT2 Large, and DeBERTa-V2 XLarge. This stark contrast underscores the importance of informed fusion decisions in optimizing model performance, as both the heuristic and GNN-based methods consistently outperformed the random strategy by significant margins on these larger models. This also demonstrates that our learned fusion strategy is effectively capturing the underlying patterns that lead to optimal fusion decisions, as it performs better than random chance.

## 6 Conclusion

This paper demonstrates that `TorchInductor`'s static, heuristic-based fusion logic can be replaced by a dynamic, data-driven cost model without compromising compilation efficiency. By integrating a semantic encoder with a Graph Attention Network (GATv2), we addressed the fundamental limitations of hand-written heuristics—specifically their hardware agnosticism, rigidity, and inability to reason over global graph structures. This work reframes operator fusion from a rule-based selection process into a learned optimization problem, capable of adapting to the non-linear data flows of modern deep learning architectures.

Our experimental evaluation across a diverse suite of 48 Hugging Face models confirms that the learned scheduler can effectively reconstruct the performance of heavily engineered heuristics (baseline), without explicit rule programming. Notably, the learned model outperformed the heuristic baseline in complex topologies such as the `DeBERTa` family (up to 18ms improvement), suggesting that graph-aware networks are particularly effective at identifying fusion opportunities in scenarios with high operator density and intricate dependency chains.

Crucially, we validated that this rigorous optimization does not come at the cost of compilation latency. The integration of an external inference server introduced negligible overhead, with compilation times remaining dominated by kernel generation rather than decision inference. This finding confirms the architectural feasibility of deploying heavy-weight neural networks within the critical path of production compilers.

While the current system primarily matches the existing performance standard, it fundamentally alters the optimization trajectory. Where static heuristics face a hard performance ceiling defined by human insight, our data-driven approach establishes a scalable foundation. Future iterations can surpass these baselines by expanding the training corpus to include a wider variance of operators, integrating low-level hardware embeddings (e.g., register pressure, shared memory usage), and extending the optimization horizon to reason about global graph efficiency rather than local pairwise fusion.

## References

[1] Tianqi Chen et al. Tvm: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[2] Zhangyin Feng et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.

[3] Tianyu Gao et al. Simcse: Simple contrastive learning of sentence embeddings. *arXiv preprint arXiv:2104.08821*, 2021.

[4] Ke Li and Jitendra Malik. Learning to optimize. *arXiv preprint arXiv:1606.01885*, 2016.

[5] Charith Mendis et al. Ithemal: Accurate, portable and fast basic block throughput estimation using deep neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[6] Unmesh Niranjan et al. Representation learning for compiler performance prediction. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2022.

[7] Petar Veličković et al. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

# Appendix

## A Training Details

The models can be found on Hugging Face at sameer-n012/cs521-gnn and sameer-n012/cs521-embedding-model

| Setting | Value |
|---|---|
| IR Encoder | CodeBERT (`microsoft/codebert-base`) |
| IR Output Dim. | 768 |
| Optimizer | AdamW |
| Loss Function | SimCSE |
| Learning Rate | $3 \times 10^{-5}$ |
| Batch Size | 16 |
| Gradient Accumulation Steps | 4 |
| Training Epochs | 5 |
| Hidden Dropout Probability | 0.2 |
| Token Dropout Probability | 0.15 |

**Table 1.** Training configuration for the IR embedding model.

| Setting | Value |
|---|---|
| Architecture | GATv2 MLP |
| GATv2 Conv Layers | 2 |
| MLP Layers | 2 |
| Edge Feature Dim. | 4 |
| Optimizer | AdamW |
| Loss Function | Pairwise Ranking Loss |
| Learning Rate | $3 \times 10^{-4}$ |
| Batch Size | 1 |
| Gradient Accumulation Steps | 4 |
| Training Epochs | 20 |
| Pairwise Ranking Loss Margin | 5.0 |
| Equal Score Regularization | 0.01 |

**Table 2.** Training configuration for the GNN decision model.